

SESSION - 09

HTL Component and dialog



Table of Contents

Create an HTL Component *2*

Exercise 1 – Creation of a content component (HTL component). **2**

Dialogs *5*

Exercise 2: Create an edit dialog **5**

Exercise 3: Modularize content by using multiple scripts **8**

Context Aware Configuration *9*

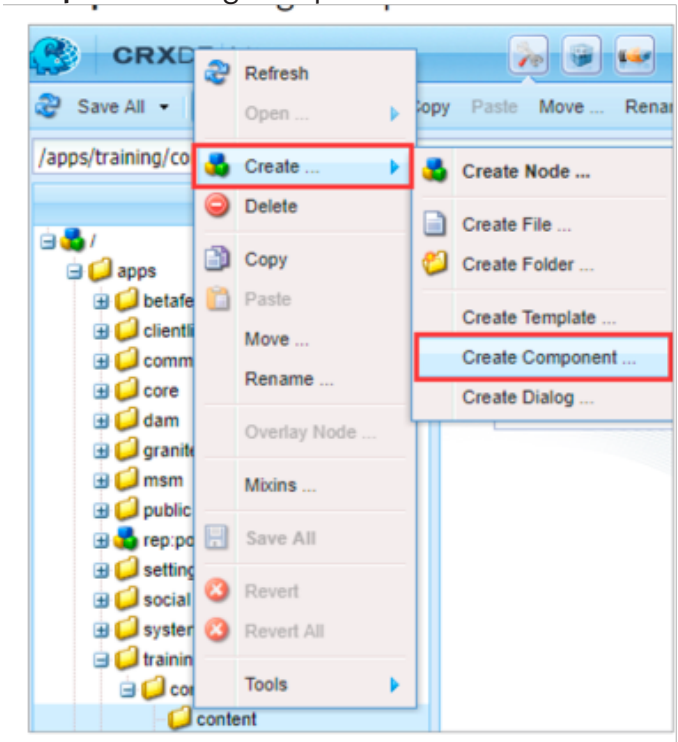
Exercise 4: Create a context-aware configuration **11**

Create an HTL Component

Exercise 1 – Creation of a content component (HTL component).

In this exercise, you will create a simple component that will be added to a page on the We.Retail website.

1. In CRXDE Lite, navigate to **/apps/training/components/content**.
2. Right-click the **content** folder and click **Create** > **Create Component**, as shown. The **Create Component** dialog box opens.



3. Enter the following details:
   a. In the **Label** field, enter **quote**.
   b. In the **Title** field, enter **Quote**.
   c. In the **Description** field, enter **My First Component**.
   d. Leave the **Super Type** field empty.
   e. In the **Group** field, enter **We.Retail** and click **Next**. The **Advanced Component Settings** are displayed.
   f. Leave the **Advanced Component Settings** blank.
   g. Click **OK**. The component is created.
   h. Click **Save All** to save the changes

NOTE: The We.Retail componentGroup was used so that the component can be added to the We.Retail website.

4. Expand the Quote component you created now by clicking the + icon next to it.
5. Right-click quote.jsp and click Rename to rename it.
6. Rename it as quote.html.

7. Double-click quote.html. The editor opens on the right.

8. Delete the existing content in quote.html and replace it with the code from start-quote.html from your exercises folder.

9. Click Save All to save the changes.

To create a dialog, where components can be added:

10. Right-click the quote component and click Create > Create Node. The Create Node dialog box opens.

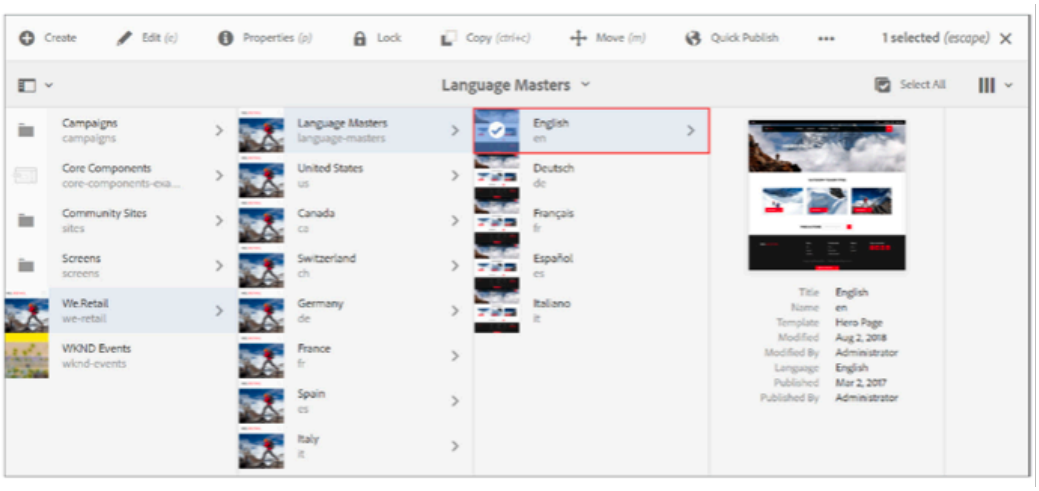
11. In the Name field, enter cq:dialog.

12. In the Type field, select nt:unstructured if it is already not selected.

13. Click OK. The node is created

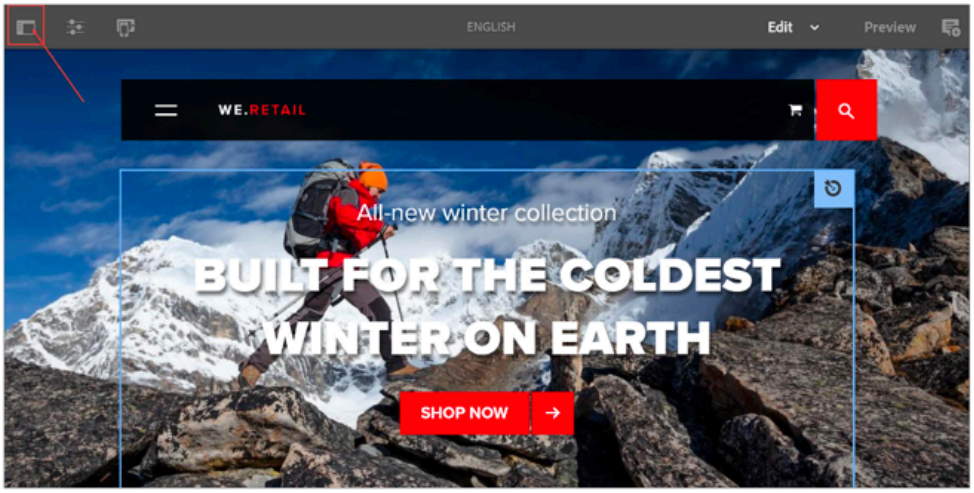
14. Click Save All to save the changes.

15. In the AEM author instance, navigate to Sites > We.Retail > Language Masters > English, as shown :

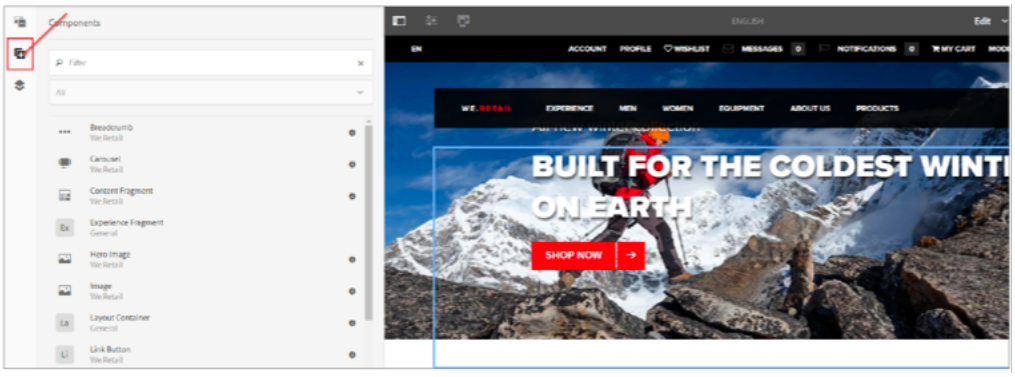


16. With the English page selected, click Edit (e) from the top menu bar to open the English page.

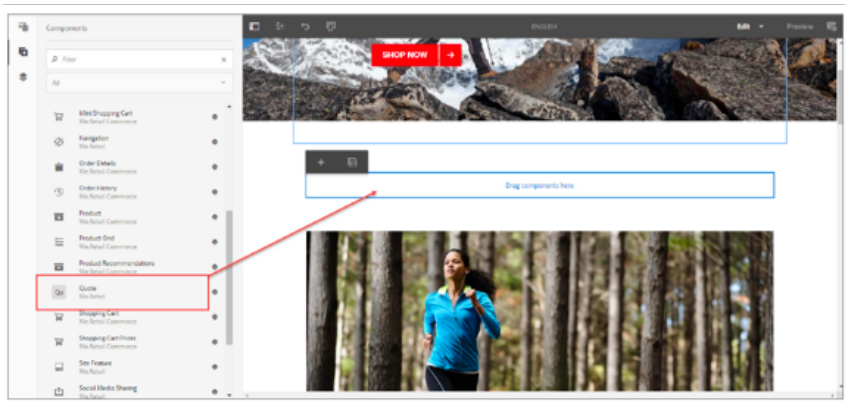
17. In the upper left, click the Toggle Side Panel icon, as shown, if you do not see the left panel.



18. In the left panel, click the Components icon, as shown. A list of available components is displayed:

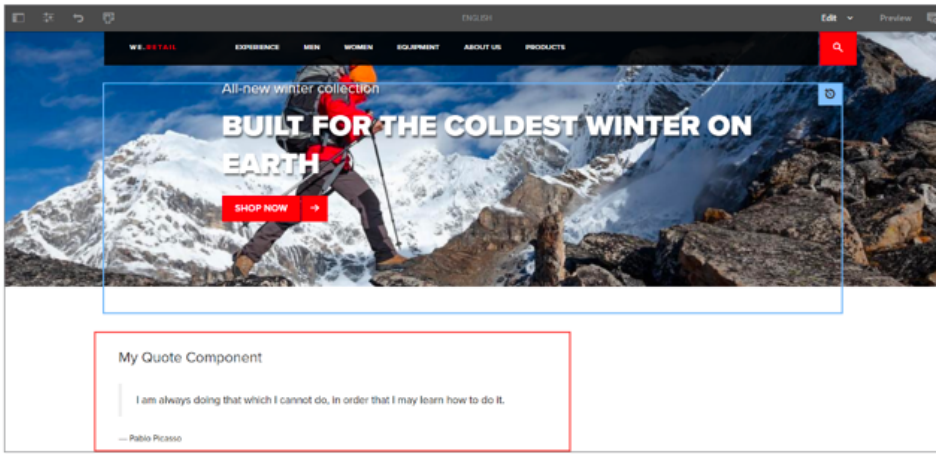


19. Look for the Quote component and drag and drop the component, as shown, onto the layout container:



NOTE: Even though the component was created in the /apps/training code base, you were still able to see and use the component on the We.Retail website. This is because the property componentGroup is We.Retail on the /apps/training/components/content/quote node.

20. Your page should look similar to the screenshot below:



Dialogs

Dialogs provide an interface for authors to configure and provide input to the component.
Depending on the complexity of the component, the dialogs can have one or more tabs. The tabs help the dialog to be short and sort the input fields.

Coral UI and Granite UI

The Coral UI and the Granite UI define the look and feel of AEM. The Granite UI provides a large range of the basic components (widgets) needed to create a dialog in the authoring environment. When necessary, you can extend this selection and create your own widget.

cq:dialog

The `cq:dialog (nt:unstructured)` node type is used to create a dialog.
The cq:dialog node:
Defines the dialog for editing the content of the component.

Is specific to the touch-enabled UI.

Is defined by using Granite UI components.

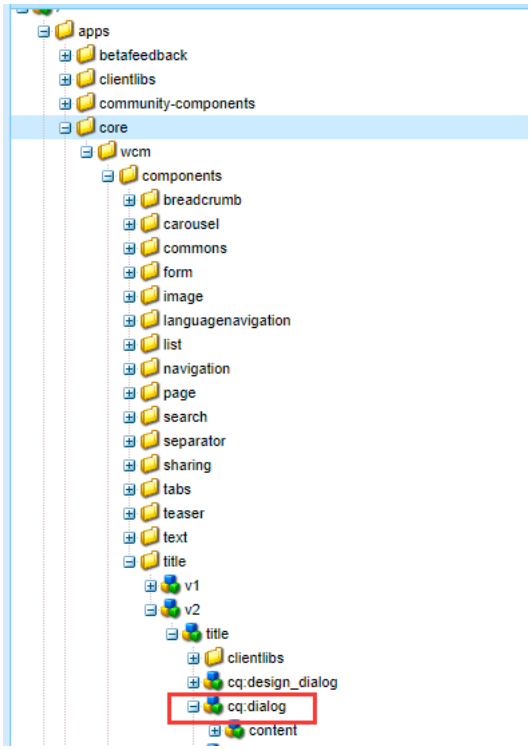
Has a property `sling:resourceType`, as the standard Sling content structure.

Can have a property `helpPath` to define the context-sensitive help resource (absolute or relative path) that is accessed when the Help icon (the ? icon) is selected. For out-of-the box components, `helpPath` often references a page in the documentation.
If no helpPath is specified, the default URL (documentation overview page) is displayed.

Exercise 2: Create an edit dialog

In this exercise, you will create a dialog box that enables an author to add a value into a form field and then have that value display on the component.

1. In CRXDE Lite, navigate to **/apps/training/components/content/quote/cq:dialog**
2. Right-click the **cq:dialog** node you created in the previous exercise and click **Delete**. The node is deleted. In this exercise, you will copy an existing dialog from a core component and modify it as per your requirement.
3. Click **Save All** to save the changes.
4. Navigate to **/apps/core/wcm/components/title/v2/title/cq:dialog**, as shown:



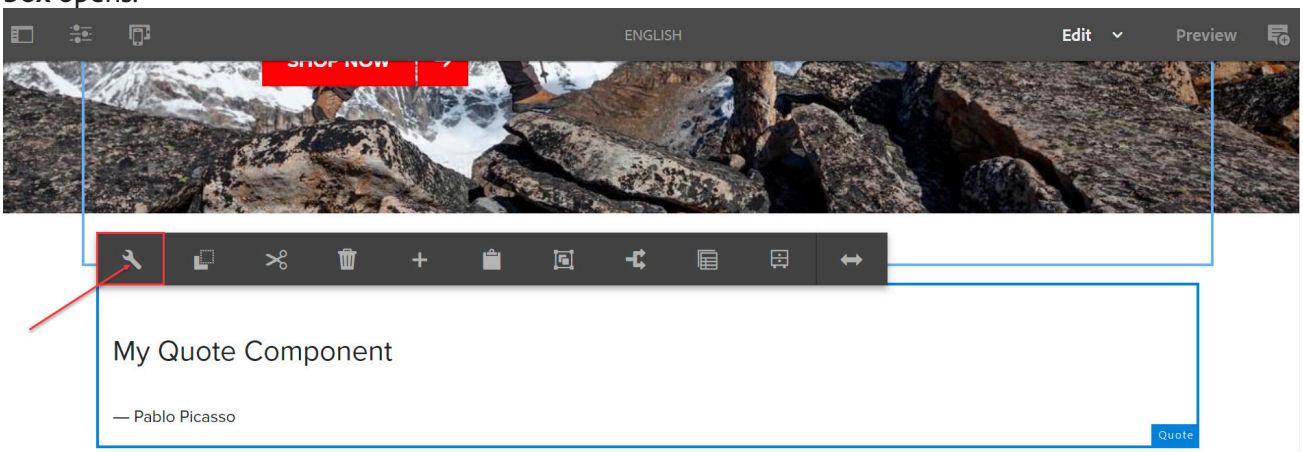
5. Right-click the **cq:dialog** node, and click **Copy**. The node is copied.
6. Navigate to **/apps/training/components/content/quote**.
7. Right-click the folder and click **Paste** to paste the node under the quote folder, as shown.
8. Select the **cq:dialog** node. You will see the properties of the cq:dialog node in the **Properties** tab on the right. You must change the dialog title as this is a different component.
9. On the **Properties** tab, double-click **jcr:title** and change the value to **Quote**.
10. Under **quote/cq:dialog/content/items/tabs/items/properties/items/columns/items/column/items**:
    a. Right-click the **types** node and click **Delete** to delete the node.
    b. Right-click the **defaulttypes** node and click **Delete** to delete the node.
    c. Right-click the **title** node and click **Rename** to rename it.
    d. Rename it as **myProperty**.
    e. Click **Save All** to save the changes.
11. Select the myProperty node to update its properties:
    a. fieldLabel: My Quote Property
    b. name: ./myQuote
    c. Click Save All to save the changes.

When an author uses this dialog and adds a value to the new formfield, AEM saves it under the current node on a property called myQuote. This value can then be accessed by the HTL script in the quote component.

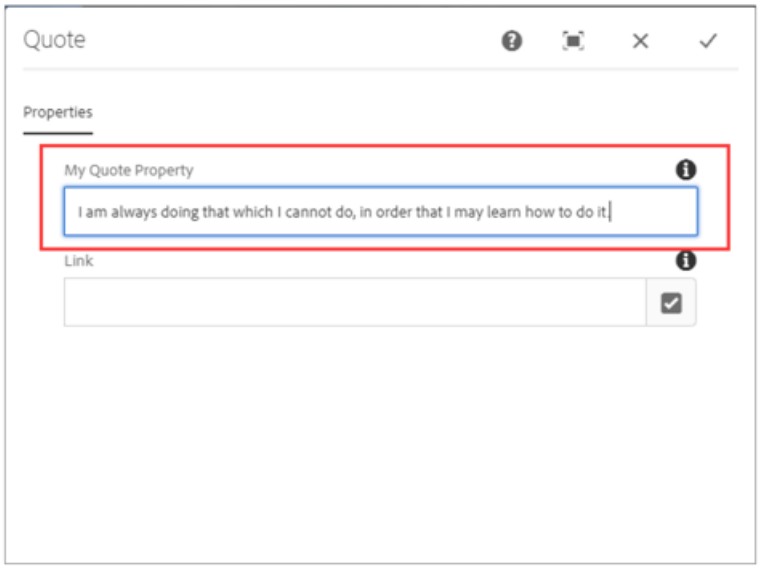
12. Double-click **quote.html**. The editor opens on the right.
13. Delete the existing content in quote.html and replace it with the code from the **dialog-quote.html** file provided to you. Notice the code `${properties.myQuote}`. By using `${}` expression,

you are able to insert an object into the HTML code. properties is a global object that gives access to all properties on the current resource.

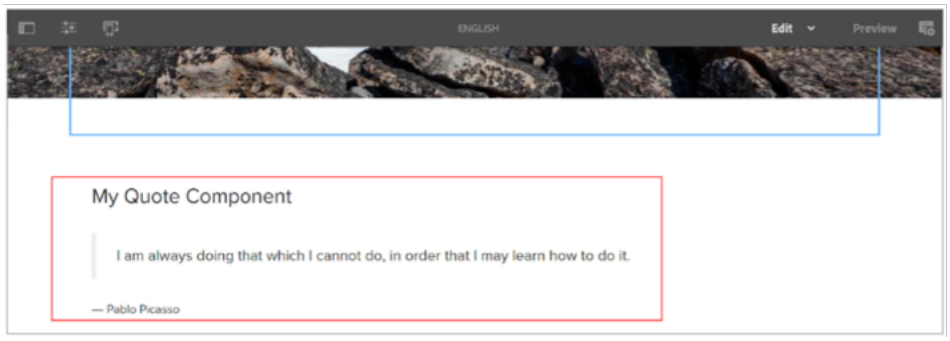
14. Click **Save All** to save the changes.
15. In the AEM author instance, navigate to **Sites** > **We.Retail** > **Language Masters** > **English**.
16. With the English page selected, click **Edit (e)** from the top menu bar to open the English page.
17. Click the **Quote** component on the page, and click the **Configure** icon, as shown. The **Quote dialog box** opens.



18. Add the following values to the dialog:
    a. My Quote Property: I am always doing that which I cannot do, in order that I may learn how to do it.
    b. Click Done (the checkmark at the top-right of the dialog).

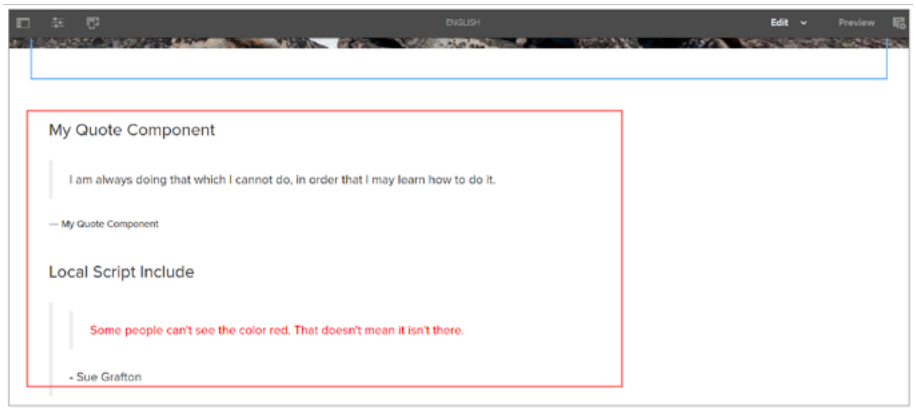


19. Observe the rendered dialog values on the page.



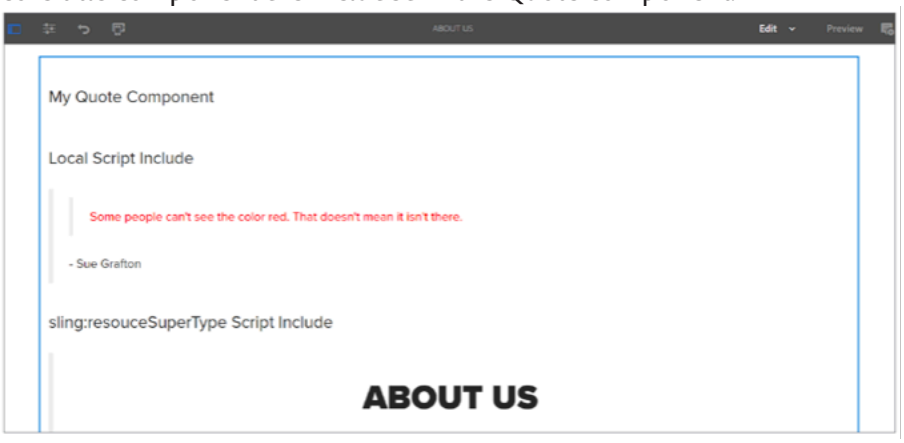
Exercise 3: Modularize content by using multiple scripts

1. In this exercise, you will modularize the components by adding local scripts to the component.
2. In CRXDE Lite, navigate to /apps/training/components/content/quote.
3. Right-click the quote component and click Create > Create File. The Create File dialog box opens.
4. in the Name field, enter red.html and click OK. The file is created.
5. Double-click red.html. The editor opens on the right.
6. Add the code from the red.html exercise file provided to you.
7. Click Save All to save the changes.
8. Open quote.html and replace the existing code with includelocal-quote.html from the exercises files provided to you.
9. Click Save All to save the changes.
10. To view the newly included script, navigate to Sites > We.Retail > Language Masters > English. Notice how both the quote.html and red.html scripts are rendered with the Quote component, as shown. The script modularization is useful in code reuse. To utilize the Core title component in the Quote component, include a title.html script that does not exist in Quote and it will automatically be included from the Title component because of the sling:resouceSuperType property on the Quote component.



11. Navigate back to CRXDE Lite. Now that you have included local scripts to the quote component, include an inherited script from the core title component.

12. Open **quote.html** and replace the existing code with the code from the **include-quote.html** exercise file provided to you.
13. Click **Save All**.
14. To view the newly included script, navigate to the **Sites** > **We.Retail** > **Language Masters** > **English** > **About Us** page. With the About Us page selected, click **Edit (*e*)** from the top menu bar to open the page.
15. Click the **Toggle Side Panel** icon.
16. From the **Components** tab on the left panel, drag and drop the **Quote** component onto the **Drag components here** area. Notice how both the quote.html and the inherited title.html script from the core title component are included in the Quote component.



Context Aware Configuration

CA-Config is a relative new feature of AEM (introduced with AEM 6.2) which stores the configuration inside the repository; but besides that, it offers some features, which are handy in a lot of use cases:

- You don't deal with nodes and resources anymore, but only with POJOs. This kind of implementation completely hides where the configuration is stored, but the location of the config is stored as property on the node.
- CA-Config implements an approach based on the repository hierarchy. If a configuration value is not found at the specified config location, the hierarchy is walked towards the root and further configurations are consulted, until a configuration is found, or the root node is hit (in that case a default value for the configuration should be used).
- the lookup process is very configurable, although the default one is sufficient in most cases.

A good example for this is content-related settings in a multi-tenant system. Imagine a multi-country and multi-language use-case (where for some countries multiple languages have to be supported); and in each country you should have different user groups, which act as approver in the content activation process. In the past this was often done by storing it as part of the pages itself, but from a permission point of view it was possible for the authors to change it themselves (which typically only causes

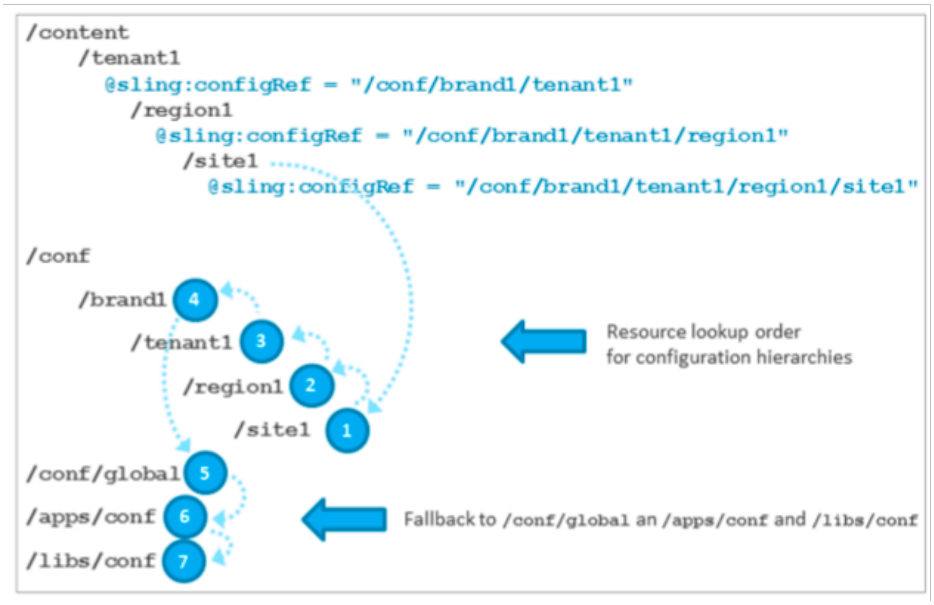
trouble). But with CA-Config this can be stored outside of the /content tree in /conf, but closely aligned to the content, and being able to match the weirdest requirements from a structural point of view; that means it is easily possible then have in one country 3 different approver groups for sections of the page, while in another just having 1 group covering all.

Another large use case are sites, which are based on MSM livecopies. There is always content which is site-specific (e.g. contact addresses, names, logos, etc). And if this content is baked into a "regular" content page which is rolled out by MSM, the blueprint content will always overwrite the content which is specific to the livecopy. With CA-config it is easy possible to maintain these site-specific configurations outside of the regular content tree.

The context-aware configuration's default implementation consists of the lookup and persistence of configuration data, resource and property inheritance, and context path detection.

The context-aware configuration implementation provides a set of Service Provider Interfaces (SPI) that helps overlay, enhance, or replace the default implementation and adapt to your needs.

The following illustration shows an example for configuration resource lookup:



The rules you must follow in resource lookup are:

- Look up the content resource tree until a resource with `sling:configRef` is found. This is the inner-most context.
- Check if a configuration resource exists at the path the property points to.
- Check for parent resources of the reference's configuration resource (below /conf).
- Look up the content resource tree for parent contexts and check their configuration resources (as they may reference a completely different location below /conf).
- Check fallback paths.

Exercise 4: Create a context-aware configuration

The Idea behind this exercise is to make you aware about the configuration browser and its feature. Since context-aware is a configuration it is also edited/updated from configuration browser console.

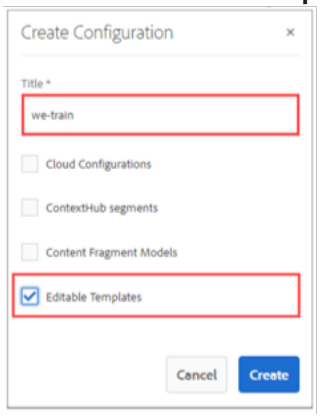
Before you create a context-aware configuration, check if you can create a template folder in the **Templates** console.

1. Click **Adobe Experience Manager** from the header bar to open the navigation pane.
2. Click the **Tools** icon and click **Templates**. The **Templates** console opens. Notice that there is no way to create a new template folder in the **Templates** console.

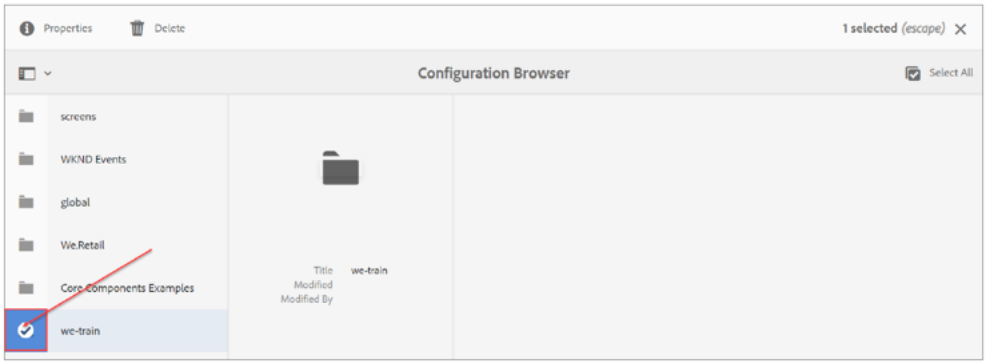


To create a new folder for editable templates, you must create a context-aware configuration.

3. Click **Adobe Experience Manager** > **Tools** > **Configuration Browser**. The **Configuration Browser** console opens.
4. Click **Create** from the actions bar. The **Create Configuration** dialog box opens.
5. In the **Title** field, enter **we-train**.
6. Select the **Editable Templates** checkbox.



7. Click **Create**. A green success message appears at the top of the page, confirming the creation of the configuration.
8. Click the thumbnail beside the **we-train** configuration, as shown, and click **Properties** from the actions bar. The **Configuration Properties** page opens.



9. Update the title to **We.Train**.
10. Click **Save & Close**. A green success message appears at the top of the page.
11. Navigate to **Adobe Experience Manager** > **Tools** > **Templates**. The **Templates** console opens. Notice that the **We.Train** folder is now available. You will use this folder in a later exercise to create an editable template.

